



Otras opciones y consideraciones legales

Como ya se mencionó, en temas anteriores se han revisado las librerías más utilizadas de Python para hacer Web *scraping*, también se revisó la manera de consumir APIs provistas por diferentes sitios, pero ahora, presentaremos una pequeña introducción a otras herramientas disponibles para llevar a cabo estas acciones.



Para empezar, hablaremos de Selenium, que es un proyecto que contiene un conjunto de herramientas y librerías que permiten y dan soporte a la automatización de los navegadores web.

Scrapy es otra herramienta popular para la extracción de información en sitios web de manera automática. A modo de analogía, trabaja como una “araña” o “robot” que navega por las páginas, encuentra la información solicitada y la organiza para su uso, resulta muy útil cuando se necesita recolectar grandes cantidades de información de internet de forma rápida y ordenada. Scrapy facilita esta tarea haciendo que el proceso sea más eficiente y menos repetitivo, y, aunque se diseñó originalmente para el Web *scraping* también puede utilizarse para extraer datos mediante el uso de una API (como por ejemplo Amazon Associates Web Services) o como un rastreador web de propósito general. Te invitamos a profundizar en estos detalles a través de la documentación oficial en las ligas que encontrarás en el material adicional.

Una de las tareas más frecuentes en la extracción de información de sitios web es para obtener texto de redes sociales, por ejemplo, de X (antes Twitter), que es una de las redes sociales más utilizadas por una gran variedad de organizaciones, celebridades, y público en general para la comunicación síncrona y asíncrona; las personas emplean X para mantenerse en contacto con otros usuarios, se utiliza un sistema de etiquetado por medio de *hashtags* que facilitan la identificación de publicaciones (antes *tweets*) por temas.



X ofrece una API que puede ser consumida utilizando una amplia gama de herramientas computacionales, donde previamente debemos registrarnos para poder hacer uso de esta, consiguiendo credenciales de acceso. En breves hablaremos de algunas librerías de Python para poder establecer esta comunicación. Y finalmente tocaremos algunos aspectos interesantes sobre el uso de estas herramientas para hacer *Web scraping*, tales como la legalidad, o la pregunta que mucha gente se hace sobre si es correcto consumir información de este tipo de sitios.

Selenium

Como mencionamos en la introducción, comenzaremos hablando de Selenium, un proyecto que proporciona extensiones para emular la interacción de los usuarios con navegadores web, este cuenta con colaboradores voluntarios quienes trabajan para que el código esté disponible para cualquier persona interesada en usarlo o mejorarlo. En el núcleo de Selenium se encuentra WebDriver, una interfaz que permite escribir instrucciones ejecutables en distintos navegadores de forma intercambiable. WebDriver es una especificación del W3C que permite interactuar con un navegador de forma nativa, ya sea localmente o de manera remota, con el objetivo de automatizar la navegación, una vez instalado y configurado todo lo necesario, basta con unas cuantas líneas de código para abrir un navegador.

En Python, esto se vería así:

```
from selenium import webdriver

driver = webdriver.Chrome()

driver.get("http://selenium.dev")

driver.quit()
```

En el siguiente ejemplo te mostramos cómo hacer *web scraping*, el código agrega algunos comentarios explicativos:

```
# importamos librerías necesarias para trabajar con Selenium

from selenium import webdriver

from selenium.webdriver.common.by import By

# La siguiente línea inicia una sesión en el navegador,

# en este caso Google Chrome, que también podemos trabajar
# con Microsoft Edge,

# Firefox, Opera, Safari, incluso el ya obsoleto Internet
# Explorer

driver = webdriver.Chrome()

# La siguiente instrucción lleva a cabo una acción,

# en este caso navegar en un sitio
```

```
driver.get("https://www.google.com")

# podemos solicitar el título de la página utilizando:

driver.title

# esto nos regresaría "Google"

# La siguiente línea, representa uno de los retos más grandes en Selenium:

# la sincronización del código con el estado actual del navegador,

# la idea principal detrás de esta instrucción es asegurarnos de que

# el elemento esté ya cargado en la página antes de que intentemos

# ubicarlo, WebDriver no lleva un registro del estado actual en tiempo

# real del DOM, por lo cual, es posible que obtengamos un error de

# No such element cuando intentemos obtener un elemento aún no cargado

# Una espera implícita no es una de las mejores soluciones a esta
```

situación, pero es un buen primer acercamiento a la solución de

la situación antes mencionada

```
driver.implicitly_wait(0.5)
```

La operación más común para interactuar con un sitio, ya sea para

solo obtener información, o para interactuar con la página, es el

encontrar elementos específicos en el DOM, pueden ser elementos de

una tabla, de una lista, elementos en un formulario, etc.

En este ejemplo, buscamos elementos por nombre, el elemento q es

la caja de texto en donde escribimos nuestra consulta en Google, y

el elemento btnK es el botón de buscar

```
search_box = driver.find_element(By.NAME, "q")
```

```
search_button = driver.find_element(By.NAME, "btnK")
```

Las siguientes instrucciones son parte de la funcionalidad de Selenium

para interactuar con el sitio, en este ejemplo, el comando `send_keys`

lo utilizamos para agregar contenido a una caja de texto (la caja de

búsqueda de google), y posteriormente mandamos el comando `click()`

al elemento `search_button` para interactuar con el sitio.

```
search_box.send_keys("Selenium")
```

```
search_button.click()
```

La siguiente instrucción busca el elemento con nombre `q`

(la caja de texto de la búsqueda) y obtiene el valor,

en este caso, `"Selenium"`

```
driver.find_element(By.NAME, "q").get_attribute("value") # => "Selenium"
```

Una vez terminada la interacción deseada con el navegador,

la instrucción `quit()` termina el proceso del Driver,

que por default, también cierra el navegador.

```
driver.quit()
```

Como lo comentamos, esto es tan solo una pequeña introducción a lo que se puede hacer con Selenium, que además de solo extraer datos, te da la posibilidad de interactuar con el sitio, esto nos abre una gran posibilidad de opciones para interactuar con diferentes portales y extraer la información que necesitamos. Si deseas profundizar en el tema, en la sección material adicional te proporcionamos algunos enlaces que puedes revisar.

Scrapy

Otra de las herramientas mencionadas en la introducción es `Scrapy`, una herramienta con un gran potencial para hacer estas tareas de extracción de información de sitios web. Veamos a continuación un ejemplo de cómo usar de una manera muy simple un “*spider*” de `Scrapy`, el siguiente fragmento de código muestra el código necesario para ejecutar una “araña” que recuperará frases de famosos de un sitio web.

```
import scrapy

class QuotesSpider(scrapy.Spider):

    name = "quotes"

    start_urls = [

        "https://quotes.toscrape.com/tag/humor/",

    ]

    def parse(self, response):

        for quote in response.css("div.quote"):

            yield {

                "author": quote.xpath("span/small/text()").

get(),
```



```

        "text": quote.css("span.text::text").get(),
    }

    next_page = response.css('li.next a::attr("href")').
get()

    if next_page is not None:

        yield response.follow(next_page, self.parse)

```

Esto debe ir en un archivo de texto, y suponiendo que lo guardamos con el nombre **quotes_spider.py**, lo ejecutaríamos con el comando **runspider** con una sintaxis como esta:

```
scrapy runspider quotes_spider.py -o quotes.jsonl
```

Cuando termine la ejecución, tendremos un archivo llamado **quotes.json** que contenga un listado de las citas en formato JSON, conteniendo el texto y autor, que se ve así:

```
{"author": "Jane Austen", "text": "\u201cThe person, be it gentleman or lady, who has not pleasure in a good novel, must be intolerably stupid.\u201d"}
```

```
{"author": "Steve Martin", "text": "\u201cA day without sunshine is like, you know, night.\u201d"}
```

```
{"author": "Garrison Keillor", "text": "\u201cAnyone who thinks sitting in church can make you a Christian must also think that sitting in a garage can make you a car.\u201d"}
```

...

Esto es tan solo el inicio de un pequeño tutorial que puedes consultar en la documentación oficial de `Scrapy`. Si te interesa profundizar más sobre el tema, en la sección de material adicional te proporcionaremos algunos enlaces para dicho fin.

Librerías de Python para web *scraping* de redes sociales

Tweepy

Uno de los usos frecuentes del web *scraping* es para extraer información de redes sociales, una de estas es X, que pese a su cambio de nombre algunos aspectos de la infraestructura se siguen denominando Twitter, por ello una librería de Python que se puede usar para estos fines de extracción de datos es Tweepy. Uno de los requisitos para emplear la herramienta es quizá la parte más tediosa, dado que para conectarnos a la API de Twitter debemos registrarnos como desarrolladores y solicitar las credenciales de autenticación (API keys). Esto se hace en el portal <https://developer.twitter.com/en>, solo ten en cuenta que debes tener una cuenta verificada en X y configurada con todos tus datos, incluyendo tu número de teléfono celular.

Para poder utilizar Tweepy, es necesario instalarlo e importarlo, así como instalar OAuthHandler, que es la librería que nos permite conectar Tweepy con nuestra cuenta de desarrollador para poder interactuar con X. OAuth es un estándar abierto que generalmente se utiliza para dar a los usuarios de sitios web o aplicaciones acceso a la información que proveen sin la necesidad de asignarles un usuario y contraseña. Este estándar es utilizado por grandes compañías como Facebook, Google, Amazon, Microsoft, X, entre otros para permitir que los usuarios compartan información sobre sus cuentas con otras aplicaciones o sitios web.

También, para trabajar con Tweepy necesitamos algunos valores de autenticación que obtenemos al completar el proceso de darnos de alta como desarrolladores en X. Si te interesa profundizar más sobre el uso de Tweepy, en la sección de material adicional te proporcionamos algunas ligas para dicho fin.

Twint

Otra librería que puedes revisar para obtener información de redes sociales es Twint, herramienta de *scraping* de X escrita en Python que permite extraer información de esta red social sin utilizar la API oficial, lo cual resulta útil si no cuentas con una cuenta de desarrollador o si simplemente no deseas crear una, ya sea por el requerimiento de datos personales u otros motivos.

Twint aprovecha los operadores de búsqueda de X para obtener publicaciones de un usuario específico, buscar resultados sobre un tema o *hashtag*, y explorar tendencias. Incluso, puede llegar a extraer información sensible, como correos electrónicos o números telefónicos. Además, permite realizar otras consultas, como ver los seguidores de un usuario, los *posts* que ha marcado con “me gusta” y a quiénes sigue.

De igual manera, si te interesa profundizar en Twint, en la sección de material adicional te dejamos algunas ligas a la documentación oficial del proyecto para que puedas revisarlas.

■ snsrape

Otra librería de Python para estos fines es snsrape, se trata de un *scraper* para servicios de redes sociales que puede extraer información como perfiles de usuario, *hashtags*, o búsquedas y regresa los elementos encontrados, tales como publicaciones relevantes. Snsrape puede trabajar con redes sociales como Facebook, Instagram, Reddit, X, entre otras. De igual forma, si te interesa profundizar en el uso de snsrape, te dejamos ligas de interés en el material adicional.

📌 Conclusión

Existen diversos proyectos cuya finalidad es la de extraer información de diferentes sitios web, particularmente de las redes sociales, al igual que de archivos con los que comúnmente trabajamos, la utilidad de cada una de las herramientas vistas dependerá de los objetivos que tengas, así como de las dificultades que representen para la simplificación de tu tarea.

📌 Consideraciones legales

Ahora, para cerrar este tema, abordaremos un punto que muchas personas consideran delicado: la legalidad del *scraping*.

A lo largo de este curso, revisamos varias opciones para recopilar información de distintos sitios web. Como se comentó en el tema de las APIs, esta es una forma “más adecuada” de hacerlo, ya que consumimos únicamente la información que

el sitio o el creador de contenido desea compartir con nosotros. Sin embargo, cuando usamos otras herramientas, también accedemos a datos que han sido publicados en algún sitio. Si están disponibles públicamente, podríamos pensar que es porque quien los publicó quiere que sean vistos o consultados. Entonces, ¿cuál es la diferencia?

Algunas empresas emergentes ven con buenos ojos el *web scraping* porque les permite obtener información sin necesidad de establecer convenios con otras compañías. No obstante, en general, a muchas empresas no les agrada que *bots* externos recopilen los datos que han publicado en su web.

La legalidad del *web scraping* depende de varios factores. Si la información está pública en Internet, se supone que está disponible para ser consultada. Pero eso no significa que podamos usarla sin restricciones. Hay aspectos éticos importantes que debemos considerar, como no hacer un número excesivo de peticiones en poco tiempo o evitar la recolección de datos personales, como números telefónicos o correos electrónicos, con fines publicitarios.

Por ejemplo, al tratarse de un proceso automatizado, podemos generar muchas solicitudes en poco tiempo. Esto puede afectar el rendimiento del servidor del sitio web. Por esta razón, incluso los sitios que ofrecen APIs gratuitas imponen límites en la cantidad de peticiones permitidas durante un periodo determinado.

Otro aspecto crucial del *web scraping* es el respeto a los derechos de autor. No debemos extraer información de un sitio y presentarla como si fuera propia. Siempre debemos citar la fuente original de la información y respetar los términos de uso del sitio del que se obtuvo, para evitar problemas legales.

Entonces, la cuestión no radica simplemente en extraer información de un sitio

web, sino en qué tipo de información se extrae, para qué se va a utilizar, si se pretende lucrar con ella, si se trata de información sensible o confidencial, o si está protegida por credenciales de acceso. Por lo tanto, el acto de extraer información de un sitio web no es ilegal en sí mismo; el problema está en cómo se hace y en todas estas consideraciones que hemos mencionado a lo largo del tema.

| Glosario

API (*Application Programming Interface*). Interfaz que permite a programas externos acceder a datos de un servicio (como redes sociales) de manera estructurada. Ejemplo: API de X (Twitter).

Autenticación OAuth. Estándar de autorización que permite a aplicaciones acceder a datos de usuarios sin compartir contraseñas. Usado por Tweepy para conectar con X.

Scrapy. *Framework* para extracción eficiente de datos a gran escala. Organiza el proceso mediante “arañas” (`spiders`).

Selenium. Biblioteca para automatizar navegadores web (Chrome, Firefox, etc.). Útil cuando los datos se cargan dinámicamente con JavaScript

snsrape. Librería para *scraping* en múltiples redes (X, Facebook, Instagram, Reddit).

Tweepy. Librería para interactuar con la **API oficial de X (Twitter)**. Requiere credenciales de desarrollador.

Twint. Herramienta para extraer datos de **X sin usar la API oficial**. Ideal para evitar límites de acceso.

Web Scraping. Técnica automatizada para extraer información de sitios web mediante código. Se usa para recolectar datos públicos (texto, imágenes, precios, etc.).

| Material adicional

JustAnotherArchivist. (s.f.). *snsrape* . GitHub. Recuperado el 30 de marzo de 2025, de <https://github.com/JustAnotherArchivist/snsrape>

Python Software Foundation. (s.f.). *twint* . PyPI. Recuperado el 30 de marzo de 2025, de <https://pypi.org/project/twint/>

Scrapy. (2025). *Scrapy at a glance* (versión 2.13.3) [Documentación]. Scrapy. Recuperado el 30 de marzo de 2025, de <https://docs.scrapy.org/en/latest/intro/overview.html>

Scrapy. (s.f.). *Scrapy documentation: Overview* [Documentación de Scrapy: Visión general]. Recuperado el 30 de marzo de 2025, de <https://docs.scrapy.org/en/latest/intro/overview.html>

Selenium. (s.f.). *Selenium documentation* [Documentación de Selenium]. Recuperado el 30 de marzo de 2025, de <https://www.selenium.dev/>

Selenium Python. (s.f.). *Selenium with Python* [Selenium con Python]. Recuperado el 30 de marzo de 2025, de <https://selenium-python.readthedocs.io/>

Taspinar, A. (s.f.). *twitterscraper* . GitHub. Recuperado el 30 de marzo de 2025, de <https://github.com/taspinar/twitterscraper>

Twitter Developer Community. (s.f.). *python-twitter documentation* [Documentación de python-twitter]. Recuperado el 30 de marzo de 2025, de <https://python-twitter.readthedocs.io/en/latest/>

Twitter Developer Community. (s.f.). *python-twitter. GitHub*. Recuperado el 30 de marzo de 2025, de <https://github.com/bear/python-twitter>

Tweepy. (s.f.). *Tweepy documentation* [Documentación de Tweepy]. Recuperado el 30 de marzo de 2025, de <https://docs.tweepy.org/en/latest/>

Tweepy. (s.f.). *Tweepy. GitHub*. Recuperado el 30 de marzo de 2025, de <https://github.com/tweepy/tweepy>

World Wide Web Consortium. (s.f.). *W3C homepage* [Página principal del W3C]. Recuperado el 30 de marzo de 2025, de <https://www.w3.org/>

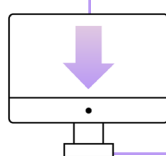
Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 30 de marzo del 2025 y su disponibilidad depende del autor.

Bibliografía

Google Translate. (s.f.). **Translate webpage:** Quotes to Scrape [Traducir página web: Quotes to Scrape]. Google Translate. Recuperado el 30 de marzo de 2025, de <https://translate.google.com/website?sl=auto&tl=en&hl=en&client=webapp&u=https://quotes.toscrape.com>

Twitter Developer Platform. (s.f.). *Twitter developer documentation* [Documentación para desarrolladores de Twitter]. Twitter Developer Platform. Recuperado el 30 de marzo de 2025, de <https://developer.twitter.com/en>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 30 de marzo del 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.